PULSE — Claude Code Handoff

Purpose: let a new Claude Code session (VS Code) continue PULSE without access to the planning chat. **Written:** 2026-10-29, about 23:45 Africa/Algiers time (UTC+01:00), by Claude in a claude.ai cloud session. **Read this first, then `CLAUDE.md`, then `IMPLEMENTATION_PLAN.md`.**

How to read the labels

Label	Meaning
[VERIFIED]	Checked by running a command or reading the code/repository in this session. Evidence is given.
[REPORTED]	Stated by Ayoub, by the team's documents, or by an external source, and not independently verified here.
[UNRESOLVED]	Not decided, not known, or blocked. Do not treat as agreed.

Facts about the code and Git were re-checked at handoff time (section 5.2), not copied from earlier chat reports.

Table of contents

1. Project conception
2. Decisions and discussion history
3. Repository and GitHub state
4. Implemented backend
5. Testing evidence
6. Files and documentation map
7. Data / AI / ML requirements
8. Frontend and technology decision
9. Remaining work, priorities and blockers
10. Team workflow
11. Exact handoff instructions for Claude Code

---

1. Project conception

1.1 The problem [REPORTED]

Hackathon problem statement "Problématique 2 — Hypertension : Détection précoce et suivi continu" (French, provided by Ayoub). Summary:

- Hypertension is often silent and poorly controlled. Follow-up relies on occasional measurements at consultations, so doctors cannot see how blood pressure changes between visits.
- Goal: improve early detection, regular follow-up, and identification of situations needing medical intervention, especially for high-risk patients.
- Suggested solution in the statement: an accessible blood-pressure monitor plus a follow-up platform that centralises readings, identifies unusual values or worrying trends, and alerts the health professional. Medical decisions stay with the professional; technology supports follow-up.
- Reasons the problem was chosen (from the team's earlier analysis, numbers not verified here): overall score 74.8/100 (second best); 7.7 million people affected, about 6.4 million uncontrolled; infrastructure compatibility 9/10 (simple tech, low-bandwidth); low dependence on electronic medical records; works urban and rural; limited extra load for professionals; measurable indicators; decision-support approach.

1.2 Product: PULSE [REPORTED as the team's goal; design decisions in section 2]

PULSE is a mobile-first web app for people with hypertension and the doctors who follow them. Patients record readings (typed in, or from a clearly labelled simulated/imported/device source). PULSE explains each reading against a target, spots worrying trends, and when a reading is dangerously high or the patient reports a red-flag symptom it switches to a short emergency flow. Every alert and emergency is saved so the patient's own doctor can review it later and leave

follow-up.

Ayoub's stated main objective: help the patient survive a possible emergency. The nearest appropriate emergency hospital is not necessarily the patient's regular clinic. The phone should show the information a bystander or responder needs. The patient's doctor reviews the incident later in the app and consults the patient afterwards. The conception must explain how this actually works, not just list it.

Product promise (proposed pitch sentence): "PULSE helps patients and the people around them know the next step when blood pressure or symptoms become dangerous, and keeps a clear record so the patient's own doctor can follow up."

Priority order (proposed in the conception; not formally approved [UNRESOLVED]):

1. Emergency action is clear, fast and reliable.
2. Readings and alerts are handled correctly.
3. Doctor follow-up and history.
4. Journal, lifestyle guidance, AI/ML extras. (Note: the later Claude Code brief made a real ML pipeline mandatory; see section 7.)

1.3 Target users

User	Goal	Key screens (planned)
Patient	Record readings, understand them, get help fast, ask the doctor questions	Home, Add reading, History, Alerts, Emergency, Journal, Profile
Bystander	Know within seconds what to do and whom to call	Emergency screen (phone unlocked); native phone Medical ID / lock-screen card
Trusted contact (family)	Know the patient needs help and where	No account in MVP; receives a phone call or a shared message
Doctor	See which assigned patients need attention, review incidents, leave follow-up	Patient list, patient detail, alert review, incident report, notes

1.4 Three paths that must never be confused

Path	Who	When	What PULSE does
Emergency services	Protection Civile / SAMU via the phone dialer	Red-flag symptoms, or severe reading with symptoms	Big call button to a configurable, team-verified number, shown first
Trusted contact	Family member chosen by the patient	Alongside or after calling emergency services	Call button + prepared share message with location; never replaces the emergency call
Regular doctor	Patient's own doctor in PULSE	Afterwards	Sees the incident, readings around it, journal; writes a note. **Not notified in real time.**

1.5 Emergency workflow (design; NOT implemented yet except the backend rules)

1. **Trigger:** red-flag symptom reported, severe reading, or manual "Urgence" button on every patient screen.
2. **Instructions:** plain French (also Arabic on the emergency screen): call emergency services now.
3. **Call:** `tel:` link to a configurable number. The phone places the call; PULSE cannot confirm it connected.
4. **Medical card + location:** name, age, key conditions, medications as entered, allergies, last reading; GPS (HTTPS + permission, ~8 s timeout) or saved home address or a manual "where am I" field. Never blocks the call button.
5. **Trusted contact:** call, or share location via `navigator.share` → `sms:` link → copy to clipboard. Logged as "share opened", never "sent".
6. **Record:** incident + action log (screen opened, call tapped, share opened/failed, location granted/denied/timeout, outcome reported). Queued offline and retried.
7. **Doctor:** reviews later; writes a note, advice, follow-up date.

Steps 1–5 must render from a local browser cache and work with no network or GPS.

Locked phone / unconscious patient (honest answer): a web app cannot show anything on a locked screen, run in the background, or detect fainting. Three layers were proposed: (1) native phone emergency info (Android "Emergency information", iPhone "Medical ID" with "Show when locked"), with a setup guide in PULSE; (2) a generated lock-screen wallpaper image with patient-chosen fields; (3) the in-app emergency screen when the phone is unlocked. (1) and (2) are "if time (high priority)"; (3) is MVP.

Loud sound: kept only as an optional tap-triggered "attract attention" button; never the main feature.

1.6 Patient and doctor experiences (planned screens)

Patient: header with red "Urgence" button and DEMO MODE banner; Home (last reading with source tag, 7-day average vs target, open alerts, "Add reading", "I feel unwell"); Add reading (systolic, diastolic, pulse, "rested 5 min", position → result card → symptom check if severe → 60 s recheck timer); Symptom check (7 checkboxes + "None of these"); History (chart with target line and markers + list); Alerts; Emergency; Journal ("ask my doctor"); Profile (health info, regular doctor, trusted contacts, emergency card fields, location consent). If time: emergency setup guide + lock-screen card; weekly general guidance cards (activity, salt, sleep).

Doctor: patient list sorted by urgency then oldest last reading; patient detail (profile summary, chart with incidents marked, summary stats, open alerts, journal, notes); alert review (reason, rule, events, Reviewed/Close with note); incident report; care-plan target; if time: AI/ML summary panel labelled as such.

1.7 Requirements and constraints [REPORTED]

- 36-hour hackathon, 3 developers: frontend, data/AI-ML, and Ayoub (backend + integration/merging).
- Must be a reliable MVP, understandable in a pitch; every feature must serve the main goal.
- Claude writes code; the team takes critical decisions; Claude may fill gaps and must flag them.
- Use fictional demo data only. Do not claim anything works unless run and verified.

1.8 What we explicitly decided NOT to promise [VERIFIED as written in docs/conception.pdf and docs/SAFETY_AND_LIMITATIONS.md]

- No automatic ambulance dispatch or emergency-centre integration; no SMS gateway.
- No real-time doctor notification (dashboard polls; pitch must not say "the doctor is alerted").
- No detection of fainting/unconsciousness; a BP cuff cannot do this; no background monitoring.
- No display on a locked phone from the web app.
- No diagnosis, no medication advice (never start/stop/change), no autonomous treatment decisions.
- No claim that a map result is the clinically best or an available hospital.
- No claim that a call/SMS/share succeeded; only that the button was tapped / composer opened.
- Simulated data always labelled; no EMR integration; no real patient data.
- ML output is never a measurement, diagnosis, emergency decision, or guarantee of safety.

---

2. Decisions and discussion history

2.1 Chronology of this chat (times are Africa/Algiers)

Time	What happened
21:20	Ayoub asked Claude to "implement the next assigned PULSE task". Claude inspected the workspace: only the problem notes, `PULSE_Claude_Conception_Handoff.md` and `PULSE_Hackathon_Conception_Report.pdf` existed — no code, no approved conception, no API contract, no tests, no assigned task. Ayoub's computer was linked but no folder was connected. Claude did not edit anything and listed the blockers.
21:42	Ayoub asked for a full, detailed PDF conception for team critique before production, noting a separate frontend dev and data/AI dev, with Ayoub doing merging and full backend. Claude checked sources (AHA crisis guidance, Algeria emergency numbers) and produced `PULSE_Final_Conception.pdf` (21 pages, v0.9, "draft for team critique").
22:14	Ayoub asked whether to connect GitHub in this chat or switch to Claude Code. Claude recommended Claude Code for implementation with GitHub as source of truth, a root `CLAUDE.md` shared by all sessions, folder ownership, and Ayoub as only merger. Claude asked whether the other two devs have Claude Code access — **not answered [UNRESOLVED]**.
22:45	Ayoub sent a "Claude Code Workspace Setup and First Implementation" brief (Phases 0–4, Git workflow, deliverables). Claude executed it in this cloud session: found the repo, cloned read-only, built and tested the first backend slice on a feature branch, committed locally, and delivered a Git bundle because push access was refused.
23:40	Ayoub asked for this handoff before switching to Claude Code in VS Code.

2.2 Changes from the team's first concept (proposed in the conception; open to critique)

First concept	Decision	Reason
Loud sound to alert people around a patient who passed out	Optional tap-triggered button only	Browser can't play sound without a tap, run in background, or detect

		fainting
Detect that the patient passed out	Out of scope	No validated sensor; a cuff doesn't detect consciousness
Emergency route to own clinic / private doctor / nearest hospital	Emergency services first; own doctor gets the incident later; nearest-hospital map link informational only (if time)	Usual clinic may have no emergency department; a map search confirms nothing
Message to ambulance with location	Out of scope; patient/bystander calls and reads out the location	No legitimate dispatch API to integrate in a hackathon
Alert to a family member chosen in advance	Kept: call + prepared share message	Works through the phone's share sheet; never claimed "sent"
Live sensor data	Manual + clearly labelled simulated; real device only if owned and tested	Bluetooth cuffs need Web Bluetooth (not on iPhone) and per-model work
Sport and food sections: do / avoid this week	If time: general education cards by weekly status; doctor advice shown separately	Personalised prescriptions are medical advice
Weekly consultation / journal	MVP: entries with "ask my doctor"; doctor marks reviewed and replies	Cheap, supports follow-up
Two dashboards	Kept: one app, two roles, one database	Both views must show the same persisted data

2.3 Decision register (mirrors `docs/decisions.md`)

ID	Decision	Status	Alternatives / reasons
D1	Backend Python 3.11+, FastAPI, SQLAlchemy 2, SQLite. Frontend React + Vite + TypeScript.	**Backend: implemented but not team-approved [UNRESOLVED]. Frontend React + TS: planned [REPORTED by Ayoub].**	Alternative: Node/Express + Prisma. Python chosen because the data/ML work and the rules share one language and the repo was empty. See section 8.
D2	Default target 140/90 until a doctor sets a care plan	Proposed, in code	Alternative 135/85 for home readings; clinician to confirm
D3	Emergency number on the main button	**[UNRESOLVED]**	14 or 1021 (Protection Civile); confirm with a medical mentor; never test-call
D4	Red-flag symptom list = AHA 7 symptoms	Proposed, in code	Clinician may extend
D5	Hosting	**[UNRESOLVED]**	Proposed: Render or Railway, FastAPI serving the built frontend, HTTPS (needed for phone GPS); backup laptop + Cloudflare tunnel
D6	UI French; emergency screen French + Arabic	**[UNRESOLVED]**	Full Arabic RTL judged too costly for 36 h
D7	Lock-screen card generator	Proposed "if time (high)"	
D8	AI/ML scope	**Changed:** conception said "doctor-only AI summary, if time"; the later brief makes a real ML pipeline **mandatory**. Target, dataset and model **[UNRESOLVED]**	See section 7
D9	Demo phone	**[UNRESOLVED]**	Proposed Android + Chrome primary, test iPhone too
D10	Real BP device available?	**[UNRESOLVED]**	Brief warns not to assume a wearable or ring measures BP
D11	Repo `Ayoub-Meziani/pulse`, `main` protected, only Ayoub merges	Repo exists [VERIFIED]; branch protection **not enabled** as far as we know [UNRESOLVED — not checkable without GitHub auth]	
D12	Contract = `api/app/schemas.py` → `contract/openapi.json`; CI checks it is current	Proposed, in code	Avoids two sources of truth
D13	No migration tool yet; schema created at startup; demo DB rebuilt by seed	Proposed, in code	Add Alembic before any data must survive a schema change
D14	Auth: seeded demo accounts, JWT	Proposed, in code	

	HS256, PBKDF2 hashes, no registration		
D15	Reading provenance: `source` ∈ manual/device/imported/simulated + `source_ref` + `external_id` (dedupe → 409)	Proposed, in code	
D16	Alert wording returned by the API in English for now	Proposed, in code	FR/AR to be added in `api/app/analysis/texts.py`

2.4 Algeria emergency numbers [REPORTED by external sources; sources disagree]

Checked 2026-10-29:

Service	Number	Source
Protection Civile	14	Widely listed; UK FCDO "Algeria: getting help" lists 14 as **fire**
Protection Civile	1021	UK FCDO lists 1021 as **ambulance**; Wikipedia "Telephone numbers in Algeria" lists 1021 as Civil Defence
Police	17 / 1548	UK FCDO
Gendarmerie	1055	Wikipedia
SAMU	varies by wilaya	Could not be verified — do not use

Rule: numbers live in configuration with a `verified` flag and source; demo mode swaps in a teammate's phone with a visible banner. **Never dial a real emergency number in testing or the demo.**

2.5 Alert thresholds source [VERIFIED by fetching the page]

American Heart Association, "Hypertensive Crisis: When You Should Call 911 for High Blood Pressure" (fetched from the goredforwomen.org copy; the heart.org URL failed to load): crisis = 180/120 or higher; wait 1–2 minutes and re-measure; call emergency services if still high with chest pain, shortness of breath, back pain, numbness, weakness, change in vision, or difficulty speaking; no symptoms → contact a health professional promptly. PULSE additionally treats any red-flag symptom as an emergency regardless of the reading (team decision from the original handoff).

2.6 Tooling discussion

- Recommendation (accepted in practice by Ayoub's next message): implement in Claude Code; GitHub is the source of truth; root `CLAUDE.md` is the shared operating contract; if a teammate uses another AI tool, copy the rules to `AGENTS.md` (not created yet).
- **[UNRESOLVED]** Do the frontend and data/AI developers have Claude Code access?

---

3. Repository and GitHub state

3.1 Remote [VERIFIED 2026-10-29 ~23:45 via `git ls-remote` and `git remote -v`]

Item	Value
Repository URL	`https://github.com/Ayoub-Meziani/pulse` (GitHub displays it as `Ayoub-Meziani/Pulse`; Git treats the name case-insensitively)
Visibility	public (from the repo listing tool)
Remote `main`	`02dc64a27b3cb5eb96cb091cdb2e84075aa1dcce` — "Initial commit" by Ayoub-Meziani, 2026-10-29T20:05:05Z (README, LICENSE MIT, Node-template `.gitignore`)
Remote branches	only `main` (and `HEAD` → `main`)
Feature branch on GitHub	**Not present.** Nothing has been pushed from this session.
Branch protection / CI on GitHub	Unknown [UNRESOLVED] — could not be checked (no GitHub auth in this session)

3.2 Local clone in the claude.ai cloud session [VERIFIED]

This clone is in Claude's cloud workspace, **not on Ayoub's computer**.

Item	Value
Path	`/home/claude/ayoub-meziani/pulse`
`origin`	`https://github.com/Ayoub-Meziani/pulse` (fetch and push)
Current branch	`feat/api-measurement-slice`
Commits	`02dc64a` Initial commit (= remote `main`) → `54937fadebadff9fddbedbf043ea679a97f5127d` "feat(api): first vertical slice - readings, validation, alert rules, doctor list" (author shown as "Claude", 2026-10-29T22:55:50+01:00) → a follow-up docs commit adding this handoff (its hash is not written here because a file cannot contain the hash of the commit that adds it; see `git log`)
Uncommitted changes	none after the handoff commit

3.3 Push-access issue [VERIFIED]

- `gh auth status` in the cloud session: token invalid (not logged in).
- Requesting **push** access to the repository from this session was **denied by the session's permission classifier**. Read access works because the repo is public. Claude did not try to work around the denial.
- Result: all work exists only locally in the cloud session and in the bundles below until Ayoub pushes it.

3.4 Bundles

Bundle	Status
`pulse-feat-api-measurement-slice.bundle`	[VERIFIED] Path in the cloud session: `/home/claude/pulse-feat-api-measurement-slice.bundle`. Size 194,012 bytes. SHA-256 `2699b2bd660d4330c7bd94a295daa66b3c04d3e3aa7c2384adde5e9f968ffeae`. `git bundle verify`: okay. Contains `refs/heads/feat/api-measurement-slice` at `54937fad…`; requires `02dc64a…`. Applied successfully to a fresh clone of the GitHub repo. Delivered to Ayoub as a file card in the chat. **It does NOT contain this handoff file.** Whether Ayoub downloaded or pushed it: [UNRESOLVED].
`pulse-feat-api-measurement-slice-handoff.bundle`	Created right after this file was committed; contains the same branch **including** the handoff commit. Its hash and verification are reported in the chat message that delivers it, not here. **Prefer this one.**

Ayoub's own computer was linked to the chat session, but no folder was ever connected and **nothing was written to it**.

---

4. Implemented backend

All of section 4 is [VERIFIED] by reading the code at commit `54937fa` and by the tests in section 5.

4.1 Directory tree (tracked files)

```
.github/workflows/ci.yml        CI: install, pytest, check contract/openapi.json is current
.gitignore                      Node template + Python/venv/DB entries
CLAUDE.md                       Shared rules for every Claude Code session
IMPLEMENTATION_PLAN.md          State, owners, next tasks, blockers
LICENSE                         MIT
README.md                       Overview, run instructions, demo accounts
PULSE_CLAUDE_HANDOFF.md         This file (added in the follow-up commit)
PULSE_CLAUDE_HANDOFF.pdf        PDF rendering of this file (added in the follow-up commit)
api/pyproject.toml              Package + deps + pytest config
api/app/__init__.py
api/app/main.py                 create_app() factory; routers under /api/v1; create_all on startup
api/app/config.py               Settings from env vars
api/app/db.py                   Engine/session factory; UTCDateTime column type; utcnow()
api/app/models.py               SQLAlchemy models (persisted contract)
api/app/schemas.py              Pydantic request/response models (API contract)
api/app/errors.py               ApiError + uniform error handlers
api/app/auth.py                 PBKDF2 hashing, JWT, get_db, current_user
api/app/permissions.py          require_patient_access, require_role
api/app/routers/auth.py         /auth/login, /me
api/app/routers/patients.py     /patients, measurements, summary, symptom-reports, alerts list
api/app/routers/alerts.py       /alerts/{id}
api/app/services/measurements.py ingest_measurement, report_symptoms, current_target
api/app/services/alerts.py      apply_evaluation (create/attach/raise + audit events)
api/app/analysis/__init__.py    Public pure functions (data/AI-owned)
api/app/analysis/types.py       Dataclasses: Target, Reading, ActiveAlert, Evaluation, Summary...
api/app/analysis/validation.py  V1–V4
api/app/analysis/rules.py       R-EMG-1, R-SEV-1/2/3, R-TGT-1, R-TRD-1
api/app/analysis/summary.py     7/30-day stats + trend direction
api/app/analysis/texts.py       All patient-facing alert wording (English only so far)
api/app/seed.py                 Synthetic demo data (python -m app.seed --reset)
api/app/export_openapi.py       Writes contract/openapi.json
api/tests/conftest.py           Temp SQLite DB, fixed clock, seeded app
api/tests/test_analysis.py      Rule vectors T1–T16 + summary tests
api/tests/test_api_slice.py     End-to-end API, permissions, errors, duplicates, seed story
contract/openapi.json           Generated API contract (20 schemas)
docs/API_CONTRACT.md            Human summary of the contract + frontend flow
docs/SAFETY_AND_LIMITATIONS.md  Rules, validation, what PULSE does not do
docs/decisions.md               Decision log
docs/conception.pdf             Full conception v0.9 (21 pages; identical to PULSE_Final_Conception.pdf, SHA-256 4465ac42…)
```

There is **no** `web/`, `ml/`, `AGENTS.md`, `CODEOWNERS`, Alembic, linter or type-checker configuration.

4.2 Architecture

- One FastAPI app built by `app.main:create_app(settings=None, clock=utcnow)`; run with `uvicorn app.main:create_app --factory`. No module-level `app` object.
- `app.state.settings`, `app.state.clock` (injectable for tests), `app.state.session_factory`.
- `Base.metadata.create_all` runs at startup (no migrations).
- Layers: routers → permissions → services → `app.analysis` (pure functions, no DB/HTTP/clock) → models.
- The backend never re-implements validation or rule logic; it calls `app.analysis`.
- ML is intentionally absent from the ingestion path (`services/measurements.py` docstring says so).
- No CORS middleware is configured (matters for the frontend; see section 8.3).
- The OpenAPI docs are served at `/api/v1/docs`; schema at `/api/v1/openapi.json`.

4.3 Endpoints (exactly the 10 operations in `contract/openapi.json`)

Base `/api/v1`. Auth header `Authorization: Bearer <token>`.

Method + path	Who	Behaviour
`GET /health`	anyone	`{"status":"ok"}`
`POST /auth/login`	anyone	body `{email, password}` → `{access_token, token_type:"bearer", user}`; email trimmed + lower-cased; 401 `invalid_credentials`
`GET /me`	any user	current user `{id, role, full_name, email}`
`GET /patients`	doctor only	assigned patients: `{id, full_name, highest_active_alert, active_alert_count, latest_measurement, last_measured_at, avg_systolic_7d, avg_diastolic_7d}`; sorted by highest active alert (emergency > severe > trend > notice > none), then oldest last reading first
`POST /patients/{patient_id}/measurements`	that patient	validate → dedupe → store → rules → alerts → 201 `{measurement, evaluation, alert, trend_alert}`

	only	
`GET /patients/{patient_id}/measurements?from&to&limit`	patient (own), assigned doctor	newest first; `limit` 1–1000 (default 200); `from`/`to` must include a timezone or 422
`GET /patients/{patient_id}/summary`	patient (own), assigned doctor	summary over the last 30 days of readings (fields in 4.7)
`POST /patients/{patient_id}/symptom-reports`	that patient only	body `{symptoms: [codes], measurement_id?, notes?}` (empty list = none) → 201 `{report, evaluation, alert}`
`GET /patients/{patient_id}/alerts?state=active|all`	patient (own), assigned doctor	default `active` = states new/acknowledged/escalated
`GET /alerts/{alert_id}`	owner, assigned doctor	alert + `events` audit list

`MeasurementIn`: `systolic` int, `diastolic` int, `pulse` int?, `measured_at` timezone-aware datetime? (defaults to server time), `source` (`manual` default | `device` | `imported` | `simulated`), `source_ref` ≤200?, `external_id` ≤200?, `context` `{rested_5_min?, position? sitting|standing|lying, arm? left|right}`.

`EvaluationOut`: `level` (none|notice|trend|severe|emergency), `rule_id`, `reason`, `next_step`, `requires_symptom_check`, `recheck_after_seconds`, `show_emergency`.

`AlertBrief`: `{id, level, rule_id, state, persistent}`. Symptom codes: `chest_pain, short_breath, back_pain, numbness, weakness, vision_change, speech_difficulty`.

Errors: always `{"error": {"code", "message", "details"}}`. Codes in use: `not_authenticated` (401), `invalid_token` (401), `invalid_credentials` (401), `forbidden` (403), `not_found` (404), `duplicate_measurement` (409, `details.existing_id`), `validation_failed` (422, list of `{field, code, message}`), `invalid_request` (422, request-shape errors), `http_<status>` (other HTTP errors, e.g. unknown route).

Not implemented (planned in `docs/conception.pdf` §11): alert transitions, incidents and actions, emergency config, emergency card, profile, trusted contacts, journal, doctor notes, care-plan editing, dev simulate/reset endpoints, ML output.

4.4 Database models (`api/app/models.py`)

All datetimes use `UTCDateTime`: stored naive UTC, returned timezone-aware UTC; naive input raises an error.

Table	Columns
`users`	id, role (patient|doctor), full_name, email (unique), password_hash, created_at
`doctor_patient_assignments`	doctor_id + patient_id (composite PK), assigned_at
`care_plans`	id, patient_id, target_systolic, target_diastolic, note, author_doctor_id, effective_from (latest effective one applies; none → default 140/90)
`measurements`	id, patient_id, systolic, diastolic, pulse, measured_at, received_at, source, source_ref, external_id, context (JSON), validation_status (valid|questionable), validation_notes (JSON), created_by; unique (patient_id, source, external_id)
`symptom_reports`	id, patient_id, symptoms (JSON list), notes, measurement_id, reported_at
`alerts`	id, patient_id, level, rule_id, reason_text, next_step_text, measurement_id, symptom_report_id, state (default new), persistent, triggered_at, created_at, updated_at
`alert_events`	id, alert_id, type (state_change|level_raised|measurement_attached|note), from_state, to_state, actor_user_id, actor_role (patient|doctor|system), note, measurement_id, created_at — append-only

Planned but **not created**: patient_profiles, trusted_contacts, incidents, incident_actions, journal_entries, doctor_notes (see conception §10).

4.5 Authentication and authorization

- Passwords: PBKDF2-HMAC-SHA256, 200,000 iterations, random 16-byte salt (stdlib; no bcrypt dependency).
- JWT HS256 with `sub` (user id), `role`, `iat`, `exp`; lifetime `PULSE_JWT_MINUTES` (default 720). No refresh tokens.

- `permissions.require_patient_access(db, user, patient_id, write=False)` is the single access check: patient → only own id; doctor → only assigned patients and only when `write=False`. Otherwise 403 `forbidden` (same answer whether or not the id exists); a valid-but-non-patient id → 404.
- `require_role(user, "doctor")` guards `GET /patients`.
- Doctors cannot create readings or symptom reports for patients.

4.6 Validation (`analysis/validation.py`)

Rule	Check	Result
V1	systolic 50–300, diastolic 30–200, pulse 25–250 (if given)	422 `validation_failed`, code `out_of_range`
V2	systolic > diastolic	422, code `not_above_diastolic`
V3	`measured_at` not more than 5 min in the future	422, code `in_future`
V4	pulse pressure < 20 or > 120	stored as `questionable` with a note; still evaluated by safety rules; excluded from averages and trend

Non-integer or missing values are rejected by Pydantic as 422 `invalid_request`.

4.7 Alert rules (`analysis/rules.py`) and summary (`analysis/summary.py`)

Constants: severe ≥180 systolic **or** ≥120 diastolic; recheck 60 s; recheck window 15 min; trend window 7 days, minimum 3 valid readings; symptom "raise" window 1 hour; trend delta ±5 mmHg systolic.

Rule	Condition	Level / action
R-EMG-1	any red-flag symptom, any reading	emergency; `show_emergency=true`; **raises** the most recent active severe/emergency alert from the last hour, else creates one
R-SEV-1	severe reading, no active severe/emergency alert triggered in the previous 15 min	severe; create; `requires_symptom_check`, `recheck_after_seconds=60`
R-SEV-2	severe reading within 15 min after an active severe/emergency alert	severe; attach to it; marks `persistent=true`
R-SEV-3	non-severe reading within 15 min after an active severe/emergency alert	attach to it (alert stays open)
R-TGT-1	reading ≥ target (either value)	notice; attach to the active notice if one exists, else create
R-TRD-1	after each reading: ≥3 valid readings in the last 7 days with mean ≥ target	trend; attach to active trend if one exists, else create
(none)	below target	no alert

Severity only rises (`level_raised` event); no rule lowers it. Wording comes from `analysis/texts.py` ("the general target" vs "your target" when a doctor set one).

Summary fields: `window_days`, `count_7d`, `count_questionable_7d`, `avg_systolic_7d`, `avg_diastolic_7d`, `avg_systolic_30d`, `avg_diastolic_30d`, `max_systolic_7d`, `max_diastolic_7d`, `pct_above_target_7d`, `days_with_reading_7d`, `trend` (rising|stable|falling|insufficient_data: last-7-day vs previous-7-day mean systolic, needs ≥3 valid readings in each), `last_measured_at`, `target {systolic, diastolic, is_default}`.

4.8 Duplicate handling, doctor view and audit trail

- Duplicates: if `external_id` is given and `(patient, source, external_id)` exists → 409 `duplicate_measurement` with the existing id. Readings without `external_id` are never deduplicated.
- Doctor view: `GET /patients` as above; doctor can read measurements, summary, alerts and alert detail of assigned patients.
- Audit: every alert creation writes `state_change` (None → new); attachments write `measurement_attached`; severity increases write `level_raised`. Only state `new` is ever produced today (no transitions yet).

4.9 Seed data (`api/app/seed.py`) — fictional, synthetic, local only

`python -m app.seed --reset` (from `api/`) drops and recreates all tables, then:

Account (email)	Role	Data
`amina@pulse.demo` — "Dr. Amina S."	doctor	assigned to Karim, Nadia, Youssef

`karim@pulse.demo` — "Karim B."	patient	rising: ~135/85 → ~152/95 over 30 days (demo patient)
`nadia@pulse.demo` — "Nadia M."	patient	stable ~128/80
`youssef@pulse.demo` — "Youssef T."	patient	stable + one 186/112 reading at 14:10 UTC three days before seeding
`samir@pulse.demo` — "Samir K."	patient	stable; **not assigned** (for permission tests)

- All accounts use the seeded demo password **`demo1234`** — deliberately documented, local development only. Never reuse it anywhere real.
- Two readings a day (08:00 and 20:00 UTC) for 30 days, `random.Random(42)`, `source="simulated"`, `source_ref="seed:<scenario>"`, `external_id="seed-<email>-<n>"`.
- Readings go through the same `ingest_measurement` path, so seeded alerts come from the rules.
- Stats are relative to seeding time: **re-seed shortly before a demo** or the 7-day windows go stale.

4.10 Configuration and dependencies

Env vars: `PULSE_ENV` (default `dev`), `PULSE_DATABASE_URL` (default `sqlite:///./pulse.db`, i.e. `api/pulse.db` when run from `api/`), `PULSE_JWT_SECRET` (an insecure placeholder is used only when `PULSE_ENV=dev`; the app refuses to start otherwise without a secret), `PULSE_JWT_MINUTES` (default 720). `.env` files are gitignored; none exist.

`api/pyproject.toml`: requires Python ≥3.11; deps `fastapi>=0.115`, `uvicorn[standard]>=0.30`, `sqlalchemy>=2.0`, `pydantic>=2.7`, `pyjwt>=2.8`; dev extras `pytest>=8`, `httpx>=0.27`.

---

5. Testing evidence

5.1 Tests that exist [VERIFIED]

39 tests: 29 in `api/tests/test_analysis.py` (rule vectors T1–T16, extra boundary/window cases, summary) and 10 in `api/tests/test_api_slice.py`: `test_health`, `test_login_rejects_wrong_password`, `test_requires_token`, `test_a1_a2_reading_persists_and_doctor_sees_it`, `test_severe_then_symptom_raises_same_alert_with_audit_trail`, `test_a4_symptom_with_normal_reading_is_emergency`, `test_invalid_reading_rejected_with_error_format`, `test_duplicate_external_id_rejected`, `test_a8_permissions`, `test_seed_story_and_summary`.

5.2 Commands run at handoff time and results [VERIFIED, 2026-10-29 ~23:45]

Environment: claude.ai cloud sandbox, Linux 6.18.44, Python 3.13.18 in a virtualenv, git 2.43.0. Installed: fastapi 0.162.0, starlette 1.10.0, uvicorn 0.54.0, SQLAlchemy 2.1.6, pydantic 2.14.2, pydantic_core 2.50.2, PyJWT 2.15.1, httpx 0.28.1, pytest 9.2.0. The package was installed with `pip install -e ".[dev]"` (from `api/`).

Command (from `api/`)	Result
`python -m pytest -v`	**39 passed, 1 warning** (Starlette deprecation: "Using httpx with starlette.testclient is deprecated; install httpx2")
`python -m app.export_openapi then git diff --exit-code -- contract/openapi.json`	no diff: contract current
`python -m app.seed --reset` (with `PULSE_DATABASE_URL` pointing at a temp file)	seeded doctor id 1, patients karim 2, nadia 3, youssef 4, samir 5
`uvicorn app.main:create_app --factory --port 8078` + curl	health ok; Nadia 186/118 → severe R-SEV-1 alert 4; symptom `chest_pain` → emergency, `show_emergency` true, same alert 4 raised to emergency; Nadia reading Samir's data → 403; doctor list order Nadia (emergency) → Youssef (severe) → Karim (trend, 2 active alerts); server log had 0 errors

5.3 Earlier report vs this verification

The earlier chat report claimed 39 passing tests, a working smoke test, a current contract, a clean secret scan and a bundle that applies to a fresh clone. At handoff time the tests, smoke test and contract check were re-run (5.2), and the secret scan was re-run over tracked files: no hard-coded credentials or token patterns (`ghp_`, `github_pat_`, `sk-`, AWS keys, private keys) apart from the documented local-only items (the dev JWT placeholder in `config.py`, the test-only secret in `conftest.py`, and the seeded demo password), and no `.env`, `.db`, `egg-info` or `__pycache__` files tracked. The fresh-clone bundle check was run when the first bundle was made (about 22:55) and is repeated for the handoff bundle when it is created.

5.4 Not tested [UNRESOLVED]

- GitHub Actions CI (never ran; nothing pushed). CI uses Python 3.12; local tests used 3.13.
- Windows (Ayoub's machine): commands, venv activation, paths.
- PostgreSQL or any non-SQLite database; concurrent writes.
- Any frontend, browser, phone, GPS, `tel:`, share, or offline behaviour (none built).
- Load/performance; `GET /patients` does one set of queries per patient (fine for a demo).
- Token expiry behaviour beyond "invalid token → 401".
- Clinical correctness of thresholds and wording (needs a clinician).

---

6. Files and documentation map

Look first: `PULSE_CLAUDE_HANDOFF.md` (this) → `CLAUDE.md` → `IMPLEMENTATION_PLAN.md` → `docs/decisions.md` → `docs/API_CONTRACT.md` → `api/app/schemas.py` → `api/app/analysis/rules.py`.

File	Purpose
`CLAUDE.md`	Ownership, git rules, contract rules, safety/honesty rules, verified commands
`IMPLEMENTATION_PLAN.md`	Current state, per-person next tasks, blockers
`docs/decisions.md`	Decision log (D1–D16)
`docs/API_CONTRACT.md`	Endpoint summary, provenance rules, frontend flow for a reading, planned endpoints
`contract/openapi.json`	Generated contract; the frontend's source for types
`docs/SAFETY_AND_LIMITATIONS.md`	Rules, validation, non-promises
`docs/conception.pdf`	Full conception v0.9 (scope table, emergency spec, data model, API v1 plan, permissions, screens, tests T1–T16 + A1–A10, demo script, 36-hour plan, risks, open decisions)
`api/app/analysis/texts.py`	All patient-facing alert wording for clinician review
`README.md`	Quick start and demo accounts

Documents from the chat **not in the repo** (they were inputs; their content is summarised in sections 1–2): `PULSE_Claude_Conception_Handoff.md`, `PULSE_Hackathon_Conception_Report.pdf` (earlier team/assistant conception, 2026-10-29), the French problem statement and team notes, and the "Claude Code Workspace Setup and First Implementation" brief.

---

7. Data / AI / ML requirements

7.1 Status

- **The "full PULSE data/AI brief" that Ayoub said he would provide has NOT been received [UNRESOLVED / BLOCKER].** Everything below comes from the Claude Code setup brief (Phase 2), the team's original notes and the conception.
- No ML code, dataset, notebook, model artefact, `docs/ML_PIPELINE.md`, `docs/DATA_DICTIONARY.md` or `docs/DEVICE_INTEGRATION.md` exists yet [VERIFIED].

7.2 Requirements from the setup brief [REPORTED by Ayoub; quoted closely]

The data/AI work must consider:

1. Database design, integrity, provenance, and safe migrations.
2. Measurement ingestion and validation, with a manual or CSV/mock adapter if real hardware is unavailable.
3. Data-quality flags and deduplication where reliable source identifiers exist.
4. Daily and weekly blood-pressure trend analytics, coverage, and insufficient-data handling.
5. Deterministic alerts with documented rules and explanations.
6. A genuine, reproducible ML training and evaluation pipeline: baseline, candidate model, leakage prevention, appropriate data splitting, metrics, saved artefacts, and inference.
7. A defensible prediction target based on the actual dataset; investigate data availability and licensing before choosing a dataset; do not fabricate training results or claim clinical validation.
8. Patient-reported food, activity, sleep, symptoms, medication adherence, and clinician-defined care plans — only where the data exists and is relevant to the MVP.
9. Structured backend contracts and a doctor-facing summary.

10. Tests, setup instructions, model documentation, safety limitations, and reproducible demo data.

Also from the brief: a real ML pipeline is **mandatory** but must be honest about its evidence; if the dataset cannot support a defensible patient-level forecast, document that and implement the strongest justified alternative. Synthetic data can demonstrate pipeline behaviour but cannot establish real-world performance. Do not assume a wearable or ring measures blood pressure; identify the hardware first, otherwise build a labelled adapter + mock/CSV path. Keep deterministic safety rules independent of ML; a forecast must never delay urgent guidance; the emergency screen and incident record must work when ML is unavailable.

7.3 Original team ideas [REPORTED]

Data view: patient history, general information (other illnesses/conditions), live sensor data overview. Lifestyle: sport and food sections with "good to do this week" and "better to avoid this week" based on the last week's state; weekly consultation/journal ("I did X and I'm worried — is it OK?"), kept for the doctor to review.

7.4 What already covers items 1–5 [VERIFIED]

Brief item	Current coverage
1 Provenance, integrity	`source`, `source_ref`, `external_id`, `created_by`, `received_at`; UTC datetimes; FK links; no migrations (D13)
2 Ingestion + validation	`POST /patients/{id}/measurements`, V1–V4. **No CSV importer yet.**
3 Quality flags + dedup	`validation_status`, `validation_notes`; 409 on repeated `external_id`
4 Trends + insufficient data	7/30-day summary, `insufficient_data` trend. **No daily aggregates or coverage-by-day series yet** (only `days_with_reading_7d`).
5 Deterministic alerts	R-* rules with reasons; 29 analysis tests
6–10	Not started

7.5 ML pipeline requirements (to implement; nothing below exists)

These are engineering requirements and **candidate** options. Target, dataset and model are [UNRESOLVED] and belong to the data/AI developer with Ayoub's approval.

- **Choose the target from real data you can legally use.** Candidate directions (not verified for licence or fit): (a) short-term forecast of a patient's next-week mean systolic from their home-reading history — needs a longitudinal home BP dataset; (b) population-level risk classification (e.g. elevated BP / hypertension) from cross-sectional public health survey data such as NHANES (US CDC) — a risk model, not a patient forecast; (c) anomaly/unusual-reading detection relative to a patient's own baseline. Datasets with credentialed access (e.g. MIMIC) are unlikely to be obtainable within 36 hours. Check each dataset's licence before use; record it.
- **Data provenance:** record dataset name, version/date, URL, licence, preprocessing steps and row counts in `docs/ML_PIPELINE.md`. Never mix synthetic rows into evaluation of real-world performance; label synthetic runs.
- **Leakage prevention:** split by patient (group split) so no patient is in both train and test; for forecasts use time-ordered splits (train on earlier windows, test on later); build features only from data available before the prediction time; fit scalers/encoders on the training split only; no target-derived features.
- **Baseline + candidate:** e.g. naive forecast (last 7-day mean) or logistic regression as baseline; a gradient-boosted or similar model as candidate. Report both.
- **Metrics:** forecasts — MAE/RMSE in mmHg vs baseline; classification — AUROC, AUPRC, calibration, with confidence intervals where feasible. Report the split sizes.
- **Artefacts:** saved model + metadata (data version, features, training date, random seed, metrics) and a script that retrains reproducibly from a clean checkout.
- **Inference contract (to agree with Ayoub before coding):** a pure function, e.g. returning `{status: "ok"|"insufficient_history"|"model_unavailable", value, interval, model_version, generated_at, limitations}`. Exposed to the **doctor view only**, labelled as a model estimate. Never feeds `analysis/rules.py`, never shown as a measurement, never required for any patient flow.
- **Location:** agree with Ayoub (`ml/` at the repo root for training, a small inference module imported by the API). Heavy dependencies (pandas, scikit-learn) should stay out of the API's core install unless inference needs them.
- **Tests:** leakage checks (no patient overlap), deterministic training on a tiny fixture, inference returns `insufficient_history` when history is short.

7.6 Device integration [UNRESOLVED]

No hardware identified. The adapter interface already exists in the contract: any device or CSV importer must POST

readings with `source="device"` or `"imported"`, a `source_ref` (device id or file name) and a stable `external_id`. A CSV import script and `docs/DEVICE_INTEGRATION.md` are still to be written. Web Bluetooth is not available on iPhone.

7.7 Honest limitations to state in the pitch

Rules are a prototype policy pending clinician review; any model is trained on the chosen dataset only and is not clinically validated; synthetic demo data shows behaviour, not accuracy.

---

8. Frontend and technology decision

8.1 Current state [VERIFIED]

No frontend code exists. The backend is Python/FastAPI and is the only implemented code.

8.2 Plan [REPORTED by Ayoub] and why the backend was not rewritten in JavaScript

- Frontend: **React + TypeScript** (Vite recommended in the conception), created by the frontend developer in `web/`.
- The backend stays Python for now because: the repo was empty and a tested slice now exists (39 tests); the mandatory ML work and the deterministic rules are naturally Python and can share one process; the OpenAPI contract lets the TypeScript frontend generate exact types, so a JS backend is not needed for type safety; rewriting would cost hackathon hours with no user-visible gain.
- **[UNRESOLVED]** The team has not formally approved D1. If the team insists on a JavaScript backend, the options are (a) keep Python as the API + ML service (recommended) or (b) port the slice and its tests to Node; decide early.

8.3 Integration notes for the frontend

- Generate types from `contract/openapi.json` (the conception suggests `openapi-typescript`; not installed yet).
- **No CORS is configured.** In development, use a Vite dev-server proxy for `/api` → `http://127.0.0.1:8000` (no backend change), or ask Ayoub to add CORS for specific origins. For deployment, the conception proposes FastAPI serving the built frontend from one origin. [UNRESOLVED which approach]
- Follow the "Frontend flow for a reading" in `docs/API_CONTRACT.md` (symptom check → emergency → recheck timer).
- Show `measurement.source` on every reading; display times in Africa/Algiers.
- The emergency screen must work offline from cached config/card; those endpoints do not exist yet.

8.4 Other unresolved architecture decisions

Migrations (Alembic or not), hosting (D5), how ML inference is packaged, where tokens are stored in the browser (conception: per-viewer only; no decision), whether the emergency config is a settings file or table (conception: settings file + env).

---

9. Remaining work, priorities and blockers

9.1 Acceptance criteria status (from the conception, A1–A10)

#	Criterion	Status
A1	Reading persists after refresh	Backend verified by test; no UI
A2	Assigned doctor sees it (within 15 s polling)	Backend verified; polling/UI not built
A3	Rules deterministic and tested (T1–T16)	**Verified**
A4	Red-flag symptom → emergency even with normal BP	Backend verified; emergency screen not built
A5	Emergency call ≤2 taps, works offline/without GPS	Not built
A6	Emergency services vs trusted contact vs regular doctor visibly separate	Not built
A7	Alerts, incidents, notes stay in history	Alerts persist with events; incidents, notes, transitions not built
A8	No user reads another patient's data	Verified for the 10 implemented operations
A9	Simulated data labelled; no unbuilt integration claimed	Data labelled in API; UI not built

A10	Demo repeats from reset	Seed reset exists; demo not built

9.2 Priorities (recommended order; Ayoub decides)

Backend (Ayoub):

1. Push the branch, open the PR, enable branch protection + CI on `main`, add `CODEOWNERS` once usernames are known.
2. Emergency backend (priority 1 of the product): `GET /config/emergency` (numbers with `verified`, `source`, demo mode + demo number from env), patient profile + trusted contacts, `GET /patients/{id}/emergency-card`.
3. Incidents: `POST /patients/{id}/incidents`, `POST /incidents/{id}/actions` (batch, idempotent on incident + action + client_time), `PATCH /incidents/{id}` (location snapshot with consent, outcome), `GET` report with timeline and readings ±24 h.
4. Alert transitions `POST /alerts/{id}/transition` with the conception §6 state machine: new→acknowledged (patient); new/acknowledged→escalated (patient/system when emergency opened, links incident); new/acknowledged→dismissed (patient, **notice only**, reason required); any open→reviewed (assigned doctor); reviewed→closed (doctor, note required for severe/emergency); anything else 409.
5. Doctor notes (private note + `advice_for_patient` + follow-up date), journal + review, care-plan PUT.
6. Dev endpoints (simulate, reset-demo) behind a flag; serve built frontend; HTTPS deploy.

Frontend: create `web/`; login; Add reading flow with symptom check and recheck timer; history chart; alerts; doctor list + patient detail; emergency screen (static from config first, then live, offline cache, `tel:`, GPS, share fallbacks, action queue); FR UI + FR/AR emergency text.

Data/AI: FR/AR strings in `texts.py`; daily aggregates/coverage if needed by the UI; CSV import adapter; ML pipeline per section 7.5; simulator scenarios for the demo.

Cross-cutting: localization (D6); migrations decision (D13); security/privacy hardening (rate-limit login if deployed, strong `PULSE_JWT_SECRET`, HTTPS, no real data, minimal emergency-card fields, location only with consent); tests for every new endpoint (permissions + state machine); demo script rehearsal from a fresh seed; backup video.

9.3 Known blockers [UNRESOLVED]

- Feature branch not on GitHub (push denied in the cloud session) → Ayoub must push from his machine.
- Full data/AI brief not received → ML target/dataset blocked.
- D1 not formally approved by the team.
- Emergency number (D3) needs confirmation before the demo.
- Teammates' GitHub usernames (for CODEOWNERS / reviewers).
- Whether teammates have Claude Code access.
- Hardware availability (D10), hosting (D5), demo phone (D9), hackathon start time / remaining hours, medical mentor availability.

---

10. Team workflow

- **People:** frontend developer (`web/`), data/ML developer (`api/app/analysis/`, future `ml/`), Ayoub — backend (`api/app/` except `analysis/`), contracts, docs, CI, deployment, and **integration lead**.
- **GitHub is the single source of truth.** Everyone clones `https://github.com/Ayoub-Meziani/pulse`.
- **Branches:** one short-lived feature branch per task (`feat/<area>-<topic>`, `fix/...`). Never commit or push to `main`.
- **Pull requests:** every change goes through a PR; CI must pass; **only Ayoub merges into `main`. No merges and no deployments without Ayoub's explicit approval.**
- **Contracts:** `api/app/schemas.py` (API) and `api/app/models.py` (DB) change only via coordinated PRs; regenerate `contract/openapi.json` in the same PR.
- **Merge cadence (conception):** checkpoints at hours 8, 16, 24; MVP feature freeze at hour 24; merge freeze at hour 34 (fixes only).
- **Every Claude Code session** reads `CLAUDE.md` first and stays in its owner's folders.

---

11. Exact handoff instructions for Claude Code

11.1 Inspect first (read-only)

```
git remote -v                # expect origin https://github.com/Ayoub-Meziani/pulse
git status --short --branch
git branch -a
git log --oneline --all -10
git ls-remote origin         # what GitHub really has
```

Then read: `PULSE_CLAUDE_HANDOFF.md`, `CLAUDE.md`, `IMPLEMENTATION_PLAN.md`, `docs/decisions.md`, `docs/API_CONTRACT.md`, `api/app/schemas.py`, `api/app/models.py`, `api/app/analysis/rules.py`.

11.2 Get the existing work onto the machine without losing anything

If `feat/api-measurement-slice` already exists on GitHub (`git ls-remote origin` shows it): `git fetch origin` and `git switch feat/api-measurement-slice`. Do not use the bundle.

Otherwise, with the bundle downloaded from the chat (prefer `pulse-feat-api-measurement-slice-handoff.bundle`):

```
# in an existing clone of https://github.com/Ayoub-Meziani/pulse whose main is 02dc64a
git bundle verify <path>/pulse-feat-api-measurement-slice-handoff.bundle
git bundle list-heads <path>/pulse-feat-api-measurement-slice-handoff.bundle
git fetch <path>/pulse-feat-api-measurement-slice-handoff.bundle feat/api-measurement-slice:feat/api-measurement-slice
git switch feat/api-measurement-slice
git log --oneline -3         # expect: <handoff docs commit>, 54937fa, 02dc64a
```

Expected: `54937fadebadff9fddbedbf043ea679a97f5127d` is an ancestor of the branch tip and the parent of the first feature commit is `02dc64a27b3cb5eb96cb091cdb2e84075aa1dcce`.

Rules: do not clone over an existing repo; do not reset, clean, force-push or delete branches; do not commit to `main`; if local changes exist, stop and ask Ayoub. **Push the feature branch only after Ayoub approves**: `git push -u origin feat/api-measurement-slice`, then open a PR to `main` for Ayoub to review.

11.3 Set up and verify (from `api/`)

macOS/Linux:

```
cd api
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
pytest                                  # expect 39 passed (1 Starlette deprecation warning)
python -m app.export_openapi            # then: git diff --exit-code -- ../contract/openapi.json  (expect no diff)
python -m app.seed --reset
uvicorn app.main:create_app --factory --reload   # http://127.0.0.1:8000/api/v1/docs
```

Windows PowerShell (not verified on Windows):

```
cd api
py -m venv .venv
.venv\Scripts\Activate.ps1              # if blocked: Set-ExecutionPolicy -Scope Process Bypass
pip install -e ".[dev]"
pytest
python -m app.seed --reset
uvicorn app.main:create_app --factory --reload
```

If results differ from section 5.2, report the difference before changing code.

11.4 Do not redo

- Do not recreate the conception, `CLAUDE.md`, the analysis rules, the 10 endpoints, the seed, or the contract export.
- Do not rename `create_app`, change the error format, or move `schemas.py`/`models.py` without Ayoub's approval.
- Do not replace the Python backend or start a second database, duplicate models, or a separate AI service.
- Do not add ML into `analysis/rules.py` or the ingestion path.

11.5 Next recommended slice (pending Ayoub's approval)

Backend slice 2 — "Emergency backend", on a new branch from the updated base (e.g. `feat/api-emergency`), after slice 1 is merged or based on it with Ayoub's agreement:

1. Emergency config: settings-backed `GET /api/v1/config/emergency` → `{country:"DZ", demo_mode, numbers:[{label, number, verified, source}], disclaimer}`; demo number from env; default `verified:false` until D3 is confirmed.
2. Minimal `patient_profiles` + `trusted_contacts` models and endpoints needed for `GET /patients/{id}/emergency-card` (name, age, key conditions, medications as entered, allergies, primary contact, last reading).
3. Incidents + idempotent action log + outcome, with permission and audit tests.
4. Update `schemas.py`, regenerate `contract/openapi.json`, update `docs/API_CONTRACT.md`, `docs/decisions.md`, `IMPLEMENTATION_PLAN.md`; add tests; report implemented / untested / blocked honestly.

Then alert transitions and doctor notes (slice 3). Coordinate field names with the frontend developer before they build the emergency screen.

End of handoff. Source of truth after this point: the GitHub repository and its PRs.